

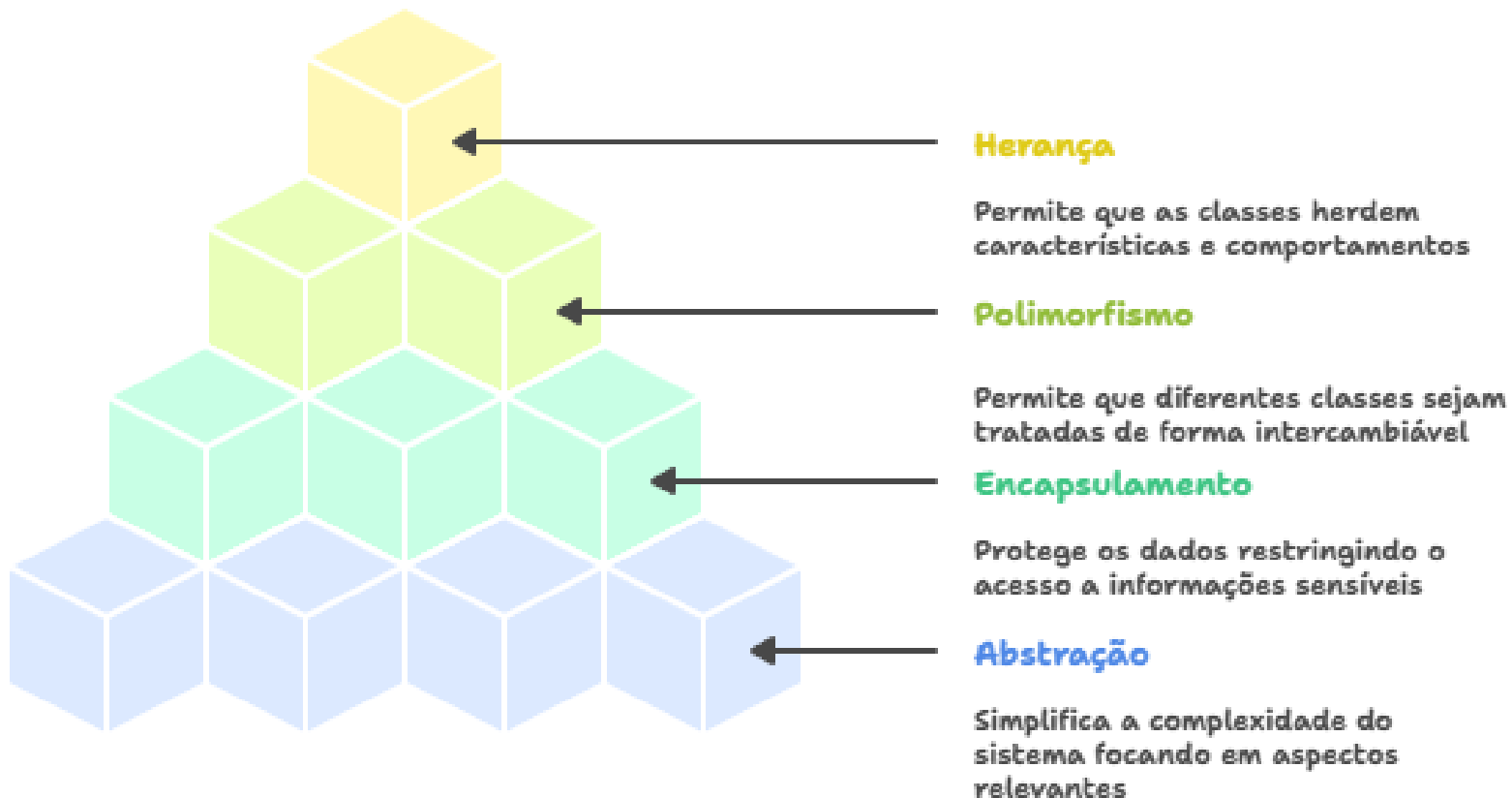
Fundamentos da Programação Orientada a Objetos

Abstração
Encapsulamento
Polimorfismo
Herança

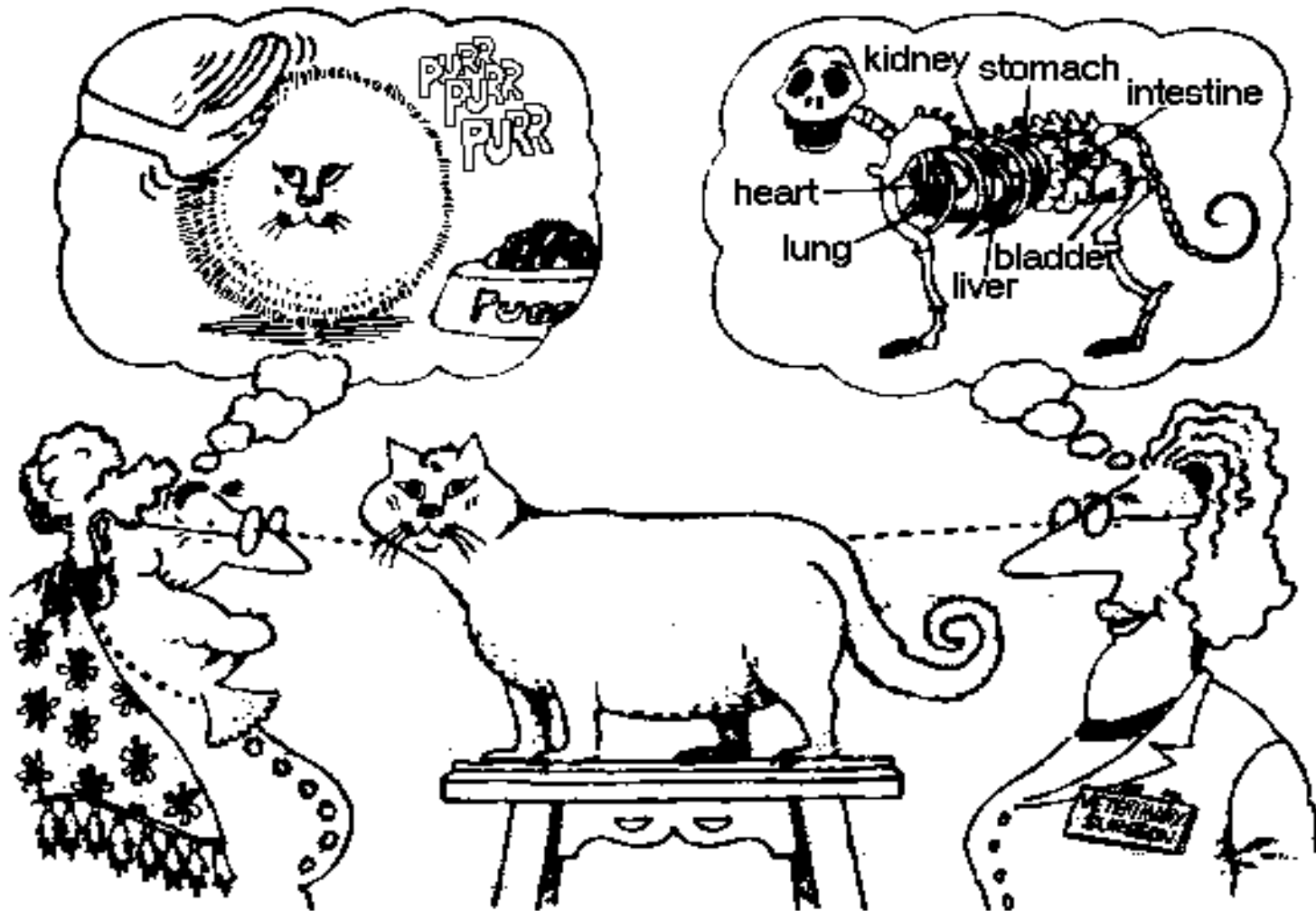


Fundamentos da Programação Orientada a Objetos

■ Pilares da Programação Orientada a Objetos



Abstração



Abstração

■ Definição 1:

*“Uma abstração denota as **características essenciais** de um objeto que o distinguem de outros tipos de objetos e assim provê uma **fronteira conceitual** firmemente definida, de acordo com a **perspectiva do observador**.”*

[Grady Booch]

Abstração

■ Definição 2:

*“(...) 2. Processo mental que consiste em **escolher ou isolar um aspecto determinado** de um estado de coisas relativamente complexo, a fim de **simplificar a sua avaliação, classificação** ou para permitir a comunicação do mesmo (...)”*

Fonte: Dicionário Houaiss

Abstração

■ Definição 3:

*“Uma abstração é uma visualização ou uma representação de uma entidade que **inclui somente os atributos de importância em um contexto particular.**”*

[Robert Sebesta]

Abstração

- Abstração:

- Técnica para lidar com a **complexidade** de um problema.
- Destaca os **aspectos importantes** do objeto real abstraído segundo o observador
- **Ignora os detalhes não relevantes** para o observador

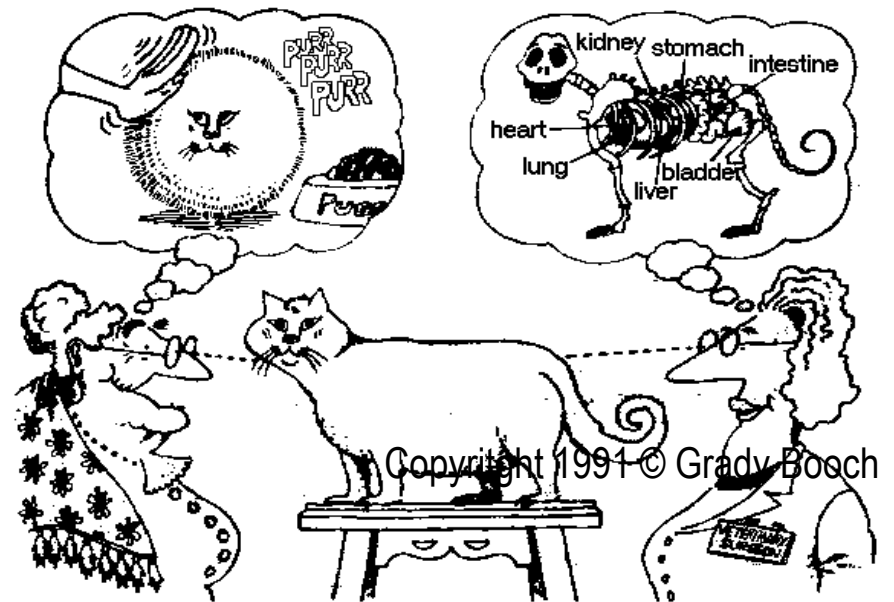
- Exemplos de uso da abstração no dia a dia:

- Dirigir um carro
- Usar um celular
- Embarcar num voo num aeroporto
- Retirar dinheiro num caixa eletrônico
- Pedir uma tapioca num restaurante
- Fazer uma ligação telefônica

Abstração

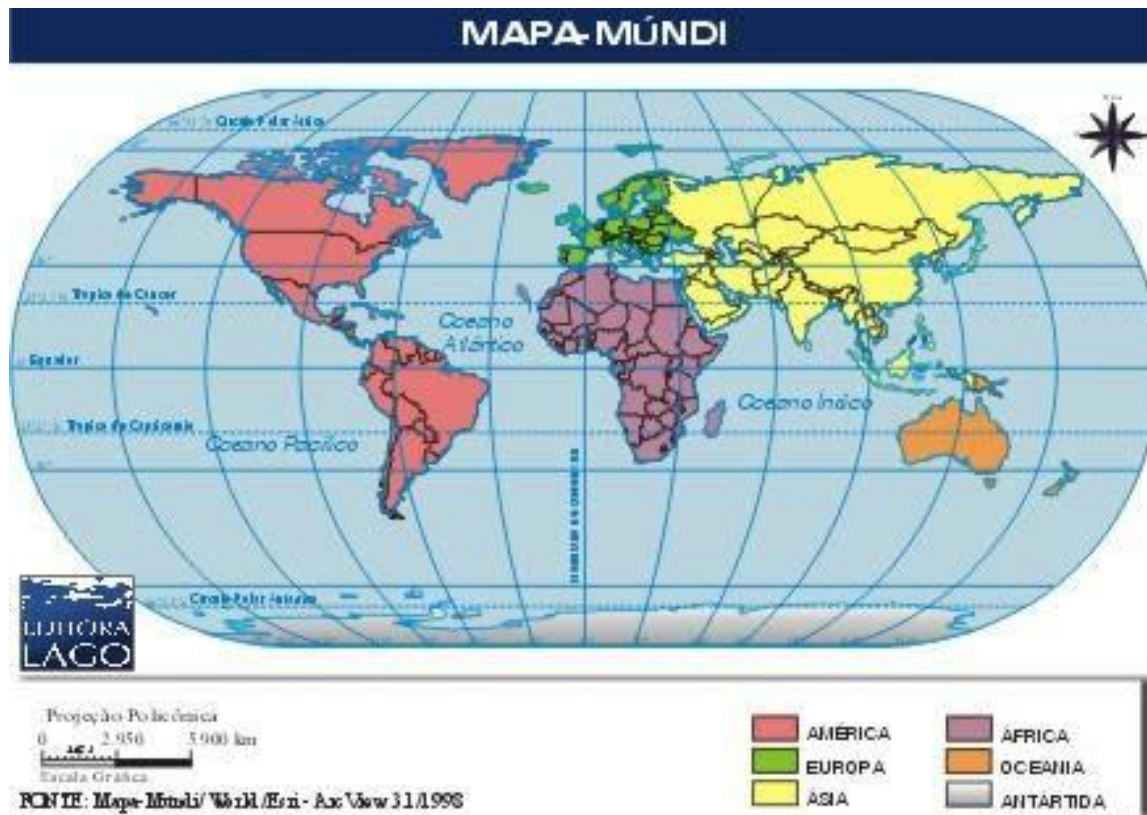
■ Exemplo de abstrações:

- Um mapa mundi
- Um aeromodelo
- Uma maquete de edifício
- Um tipo de dado abstrato (pilha)
- Uma planta baixa de uma casa
- O povo das cavernas de Platão
- Um programa de computador
- Uma pintura abstrata
- Uma ficha de paciente de hospital



Exemplos de Abstrações

Isto não é o mundo!



Exemplos de Abstrações

Não são aviões alemães da segunda guerra!



Exemplos de Abstrações

Não são casas! →

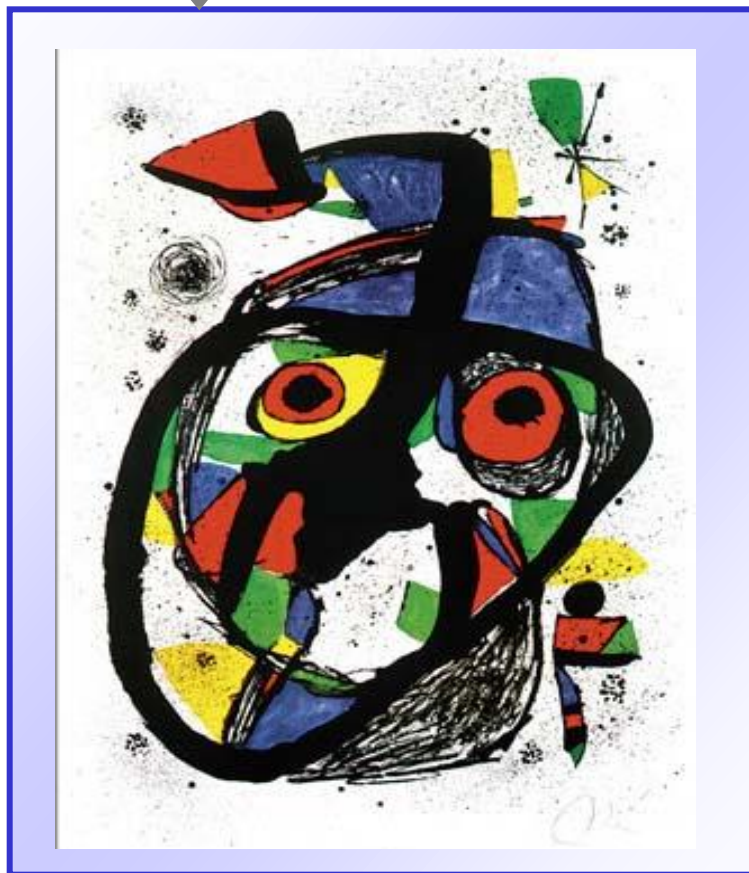


Solar dos Teles



Exemplos de Abstrações

Não é uma face (manga? coração?)!



Abstração

■ Abstração na programação:

■ Abstração de procedimentos:

- Dividimos um programa em **subprogramas** menores e mais fáceis de escrever e compreender ("dividir para conquistar").
- Para usar um subprograma escrito por terceiros abstraímos a sua implementação e nos concentramos na sua interface.
- Você precisa saber como o Python implementa a função **print()** para usá-la?

■ Abstração de dados:

- Serve para representarmos entidades reais do domínio do problema numa linguagem de programação, identificando as **propriedades** destas entidades que interessam ao sistema bem como suas **operações**.
- Ao ser matriculado no IFPB, o instituto criou uma **abstração sua chamada "aluno"** e registrou isso nos sistemas informáticos como um conjunto de dados seus que **são de interesse do IFPB** (nome, cpf, data de nascimento, nota do Enem, nomes dos pais, endereço etc.) e **ignorou os que não interessam** (cor do seu cabelo, tamanho do seu pé, tipo de comida favorita, nome da sua prima/primo, data em que você foi ao São João de CG pela primeira vez etc.)

Abstração de procedimentos

- Quem usa as funções abaixo interessa-se por:

```
import math

valor = "4"
num = int(valor)
resultado = math.sqrt(num)
print(resultado)
```

- Saber o nome das funções (ex: para calcular raiz quadrada é **math.sqrt**)
- Saber quantos são os parâmetros para as funções e em que ordem são passados
- Saber quais os tipos destes parâmetros
- Saber os tipos dos dados que as funções retornam
- Saber o significado desse dado retornado

Abstração de dados

- Define-se uma estrutura e operações para um novo tipo:

```
typedef struct st_stack {  
    int top;  
    float data[MAX];  
} stack;  
  
stack* newstack (int) {...}  
void push (stack *, float) {...}  
float pop (stack *) {...}  
int isempty (stack) {...}  
float top (stack) {...}  
void destroy (stack *) {...}
```

Exercício: abstraindo um conceito do mundo real

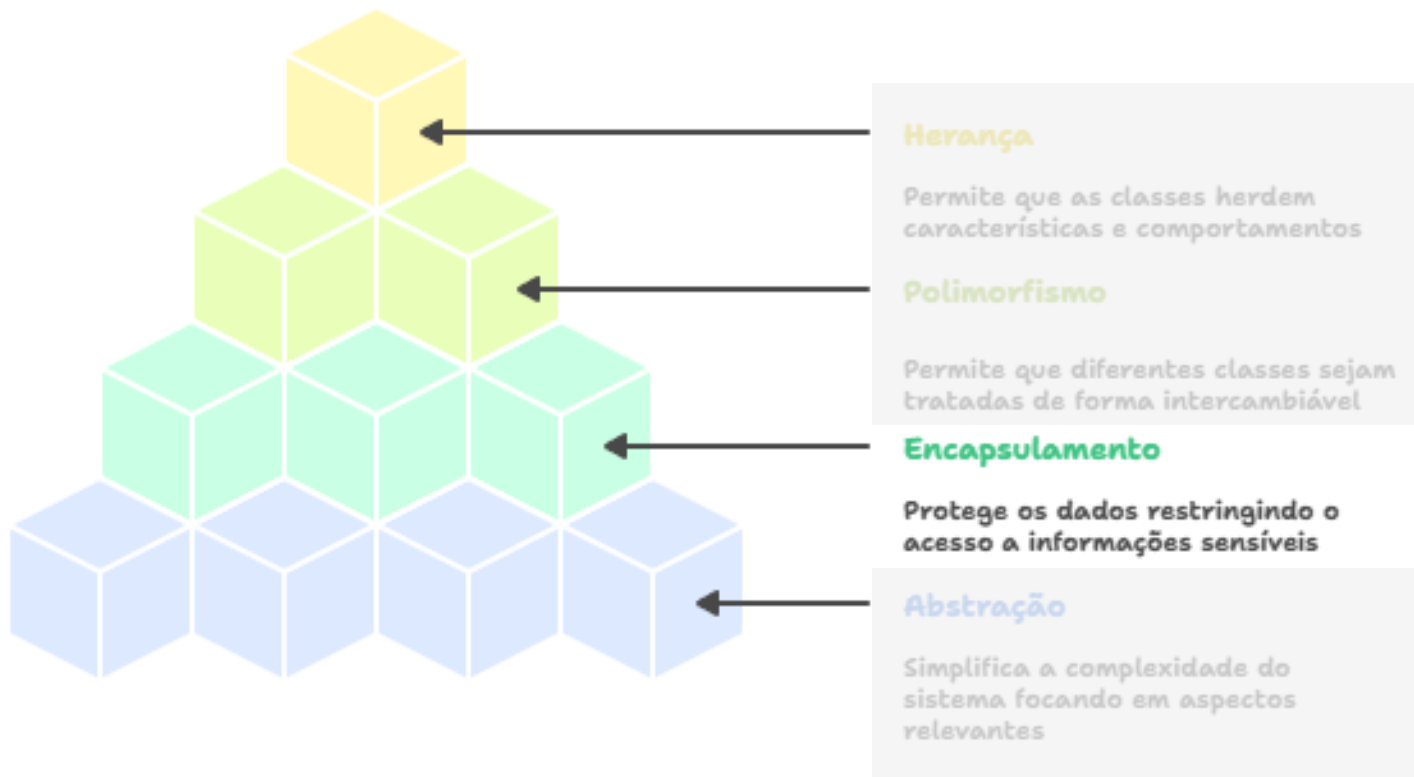
Enunciado

Imagine que você foi contratado para **criar uma abstração** de algum aspecto do mundo real.

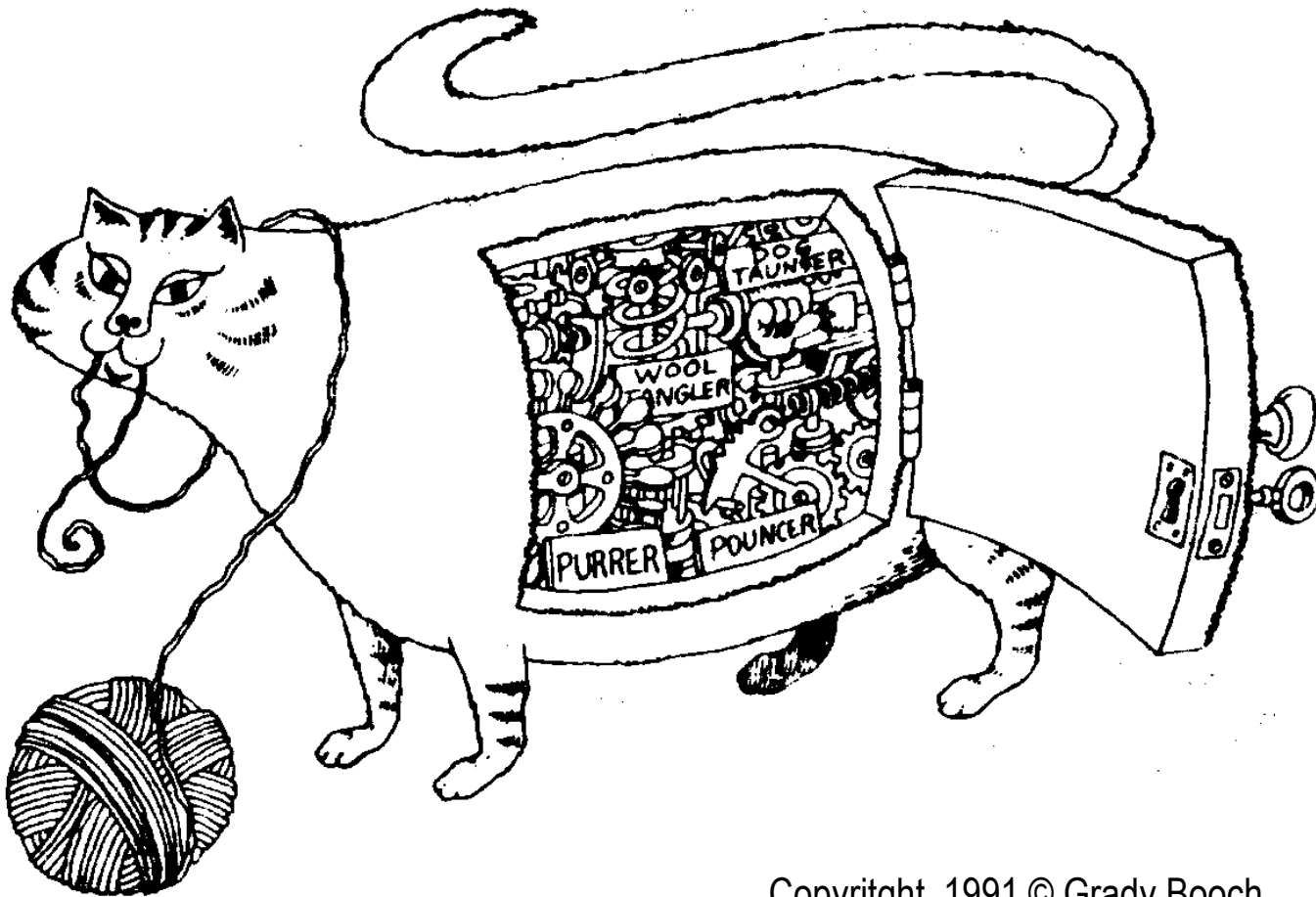
1. Escolha **um objeto ou conceito do cotidiano** (ex.: Carro, Livro, Conta Bancária, Pessoa, Animal de Estimação, Smartphone).
2. Identifique pelo menos **5 propriedades** (atributos) que descrevem as características principais dessa entidade.
 - Exemplo: para um **Carro** → cor, modelo, ano, velocidade atual, ligado.
3. Identifique pelo menos **3 comportamentos** (métodos) que representem ações típicas do objeto.
 - Exemplo: para um **Carro** → ligar, acelerar, frear.
4. Explique **por que você escolheu esses atributos e métodos**, justificando a abstração realizada.

Encapsulamento

Pilares da Programação Orientada a Objetos



Encapsulamento



Copyright 1991 © Grady Booch

Encapsulamento

■ Definição 1:

*“**Encapsulamento** é o processo de esconder todos os detalhes de um objeto que não contribuem para suas características essenciais.”*

[Grady Booch]

Encapsulamento

■ Definição 2:

*“O encapsulamento é um **agrupamento de subprogramas e dos dados** que eles manipulam. O **encapsulamento**, separada ou independentemente compilável, constitui um **sistema abstraído** e uma organização lógica para uma coleção de computações relacionadas.”*

[Robert Sebesta]

Encapsulamento

- Considerações/recomendações:

*“**Nenhuma parte** de um sistema complexo **deve depender** dos **detalhes internos** das outras partes”*

*“Para uma **abstração** dar certo sua **implementação** deve estar **encapsulada**.”*

[Grady Booch]

Encapsulamento

■ Encapsulamento:

- Esconde os detalhes internos de uma abstração
- Uma vez selecionada a implementação de uma abstração, ela deve ser tratada como um segredo da abstração e escondida dos seus clientes.

■ Encapsulamento *versus* abstração

- A abstração representa um conceito do domínio do problema, com suas características essenciais identificadas
- O encapsulamento impede que os detalhes internos de como você implementou esta abstração sejam conhecidos de quem vai usá-la
- Você não precisa acessar componentes internos de um relógio para usá-lo, todos os mecanismos que fazem os ponteiros se moverem com precisão estão encapsulados pelo corpo do relógio. Apenas os controles essenciais são expostos (botões para ajuste dos ponteiros, os próprios ponteiros, botão para ajuste da data, etc)

Encapsulamento

- Exemplos de encapsulamentos:
 - Encapsulamentos de circuitos integrados (*chips*)
 - O gabinete do seu microcomputador
 - O encapsulamento do seu smartfone
 - O corpo de um datashow
 - Nossa pele

Encapsulamento

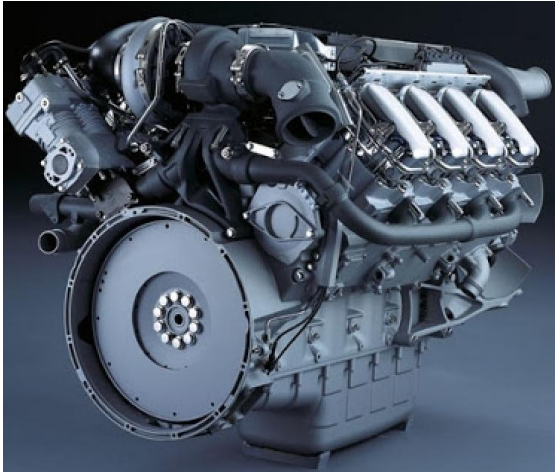


encapsulado



exposto

Encapsulamento



encapsulado



exposto

Encapsulamento



encapsulado



exposto

Encapsulamento



encapsulado



exposto

Encapsulamento

■ Encapsulamento nas linguagens:

- Em C, funções são completamente encapsuladas, mas a estrutura de um tipo abstrato de dados não é completamente protegida (encapsulamento imperfeito)
 - Funções em outras bibliotecas têm acesso à estrutura interna do tipo, violando o encapsulamento
- Em Modula-2 há como encapsular totalmente tanto dados como subprogramas
- Java e C++ possuem construções sintáticas para controlar o nível de visibilidade de um elemento (tipos, dados ou subprogramas) numa biblioteca

Encapsulamento

■ Exemplo em C:

Elementos **visíveis** fora da biblioteca.

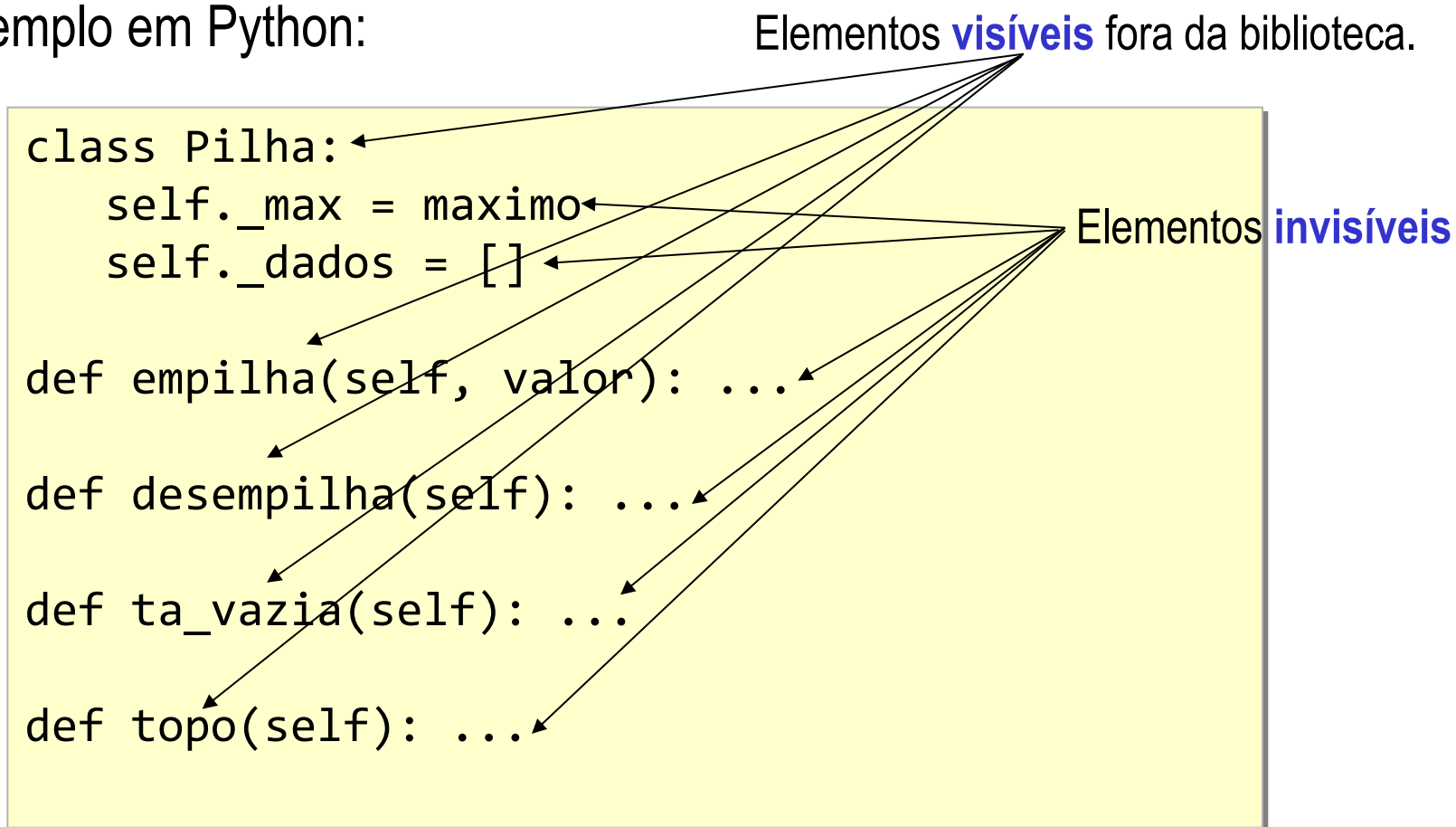
```
typedef struct st_pilha {  
    int topo;  
    float dados[MAX];  
} pilha;  
  
void cria_pilha (pilha *) {...}  
int empilha (pilha *, float) {...}  
int desempilha (pilha *, float *) {...}  
int ta_vazia (pilha) {...}  
int topo (pilha, float*) {...}  
int limpa (pilha *) {...}
```

Elementos **invisíveis**

Obs: Em C, um arquivo de biblioteca define uma fronteira de encapsulamento.

Encapsulamento

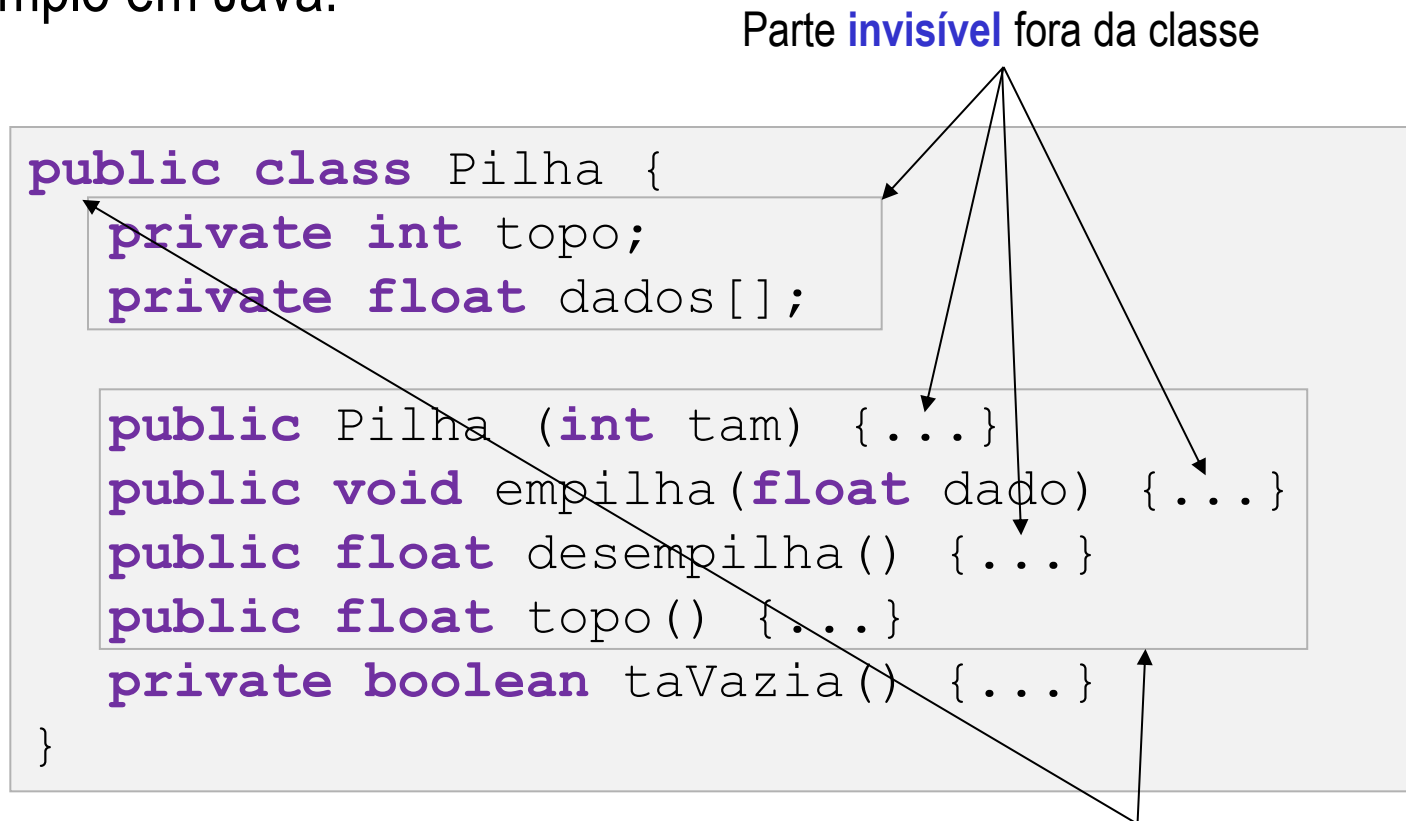
■ Exemplo em Python:



Obs: Em Python, a classe é a forma de modularizar uma entidade (tipo+operações)

Encapsulamento

■ Exemplo em Java:



Obs: Em Java, uma classe define uma fronteira de encapsulamento.

Tipos Abstratos de Dados

- Quando a linguagem não oferece um tipo nativo adequado para representar uma entidade do domínio do problema recorremos a um TAD
- *"Um TAD é um encapsulamento que inclui somente a representação de dados de um tipo específico de dado e os subprogramas que fornecem as operação para esse tipo"* [Sebesta99]
- $\text{TAD} = \text{encapsulamento}(\text{estrutura} + \text{operações})_{\text{TIPO}}$

Tipos Abstratos de Dados

■ Benefícios da programação com TADs:

- Método para organizar melhor um programa em unidades lógicas que podem ser compiladas em separado.
- Permite modificações localizadas na representação ou operações do tipo (promove a manutenibilidade)
- O encapsulamento promove a independência entre o TAD e seus clientes.
- Aumento da confiabilidade, uma vez que os clientes não têm acesso direto à representação do TAD, a não ser por suas operações.

Tipos Abstratos de Dados

- Um TAD **pilha** implementado com arranjos:

```
typedef struct st_pilha {  
    int topo;  
    float dados[MAX];  
} pilha;
```

Estado do TAD pilha

```
void cria (pilha *) {...}  
void empilha (pilha *, float) {...}  
float desempilha (pilha *) {...}  
int ta_vazia (pilha) {...}  
float topo (pilha) {...}  
void limpa (pilha *) {...}
```

Interface do TAD pilha (API)

Tipos Abstratos de Dados

- Um cliente para o TAD **pilha**:

```
main() {  
    pilha p;  
    float x, y;  
  
    cria(&p);  
    if (ta_vazia(p)) {  
        empilha(&p, 1.2f);  
        empilha(&p, 3.0f);  
    }  
    x = topo(&p);  
    if (x == 3.0f)  
        y = desempilha (&p);  
    limpe(&p);  
}
```

Em respeito ao encapsulamento, não há acessos à representação da pilha nos clientes, mas apenas às suas operações.

Tipos Abstratos de Dados

- A implementação muda para lista encadeada:

```
typedef struct st_pilha {  
    float dado;  
    struct st_pilha prox;  
} nopilha;  
typedef nopilha* pilha;
```

```
void cria (pilha *){...}  
void empilha (pilha *, float) {...}  
float desempilha (pilha *) {...}  
int ta_vazia (pilha) {...}  
float topo (pilha) {...}  
void limpa (pilha *) {...}
```

Modifica-se
a representação do TAD

E a implementação
das operações

Tipos Abstratos de Dados

- O cliente não se altera (compare ao anterior):

```
main() {  
    pilha p;  
    float x, y;  
  
    cria(&p);  
    if (ta_vazia(p)) {  
        empilha(&p, 1.2f);  
        empilha(&p, 3.0f);  
    }  
    x = topo(&p);  
    if (x == 3.0f)  
        y = desempilha(&p);  
    limpe(&p);  
}
```

Tipos Abstratos de Dados

- Entretanto, se o encapsulamento fosse violado...

```
main() {  
    pilha p;  
    float x;  
  
    cria(&p);  
    if (p.topo == -1) {  
        empilha(&p, 1.2f);  
        empilha(&p, 3.0f);  
    }  
    x = p.dados[p.topo];  
    if (x == 3.0f)  
        y = desempilha (&p);  
    limpe(&p);  
}
```

Acessos à estrutura
do TAD violam o encapsulamento e
aumentam o acoplamento entre
o TAD e seus clientes

Testar se topo = -1 é como se testa se a pilha
de array é vazia, mas não serve para a pilha
de lista encadeada. Neste caso, o teste é se
o topo = null

Retirar o elemento do topo com a
implementação de arrays é indexar o array
com topo, mas para a pilha com lista
encadeada é mudar o ponteiro do topo para
o próximo nó

TADs e o encapsulamento

“A habilidade de **mudar a representação** de uma abstração **sem perturbar quaisquer de seus clientes** é o principal benefício do **encapsulamento**”

“O **encapsulamento não impede** o programador de fazer **coisas estúpidas**”

[Grady Booch]

“O **encapsulamento previne acidentes**, não **fraudes**”

[Bjarne Stroustrup]

Tipos Abstratos de Dados

- O conjunto de operações públicas que um TAD oferece é chamado de *Interface do TAD* ou *API (Application Programming Interface)*.
- Quando é que uma mudança num TAD exige mudanças nos clientes?
 - Quando há mudanças na **interface (api)** dele.
 - Se a interface permanecer a mesma, os clientes não se alteram.
 - Exemplo: se a função topo mudar para topo(pilha p, int index), o cliente teria que ser modificado.

Programação Orientada a Objetos

- Em geral, os TADs definidos numa linguagem orientada a objetos são chamados de **classes**.
- Instancias particulares de uma classe são chamados **objetos**.
- Uma classe contém um **estado encapsulado** e uma **interface pública** de operações
- Para solicitar ao objeto a execução de uma operação, enviamos-lhe uma **mensagem**
- Classes podem ser **especializadas** por meio da **herança**
- Classes podem ser **relacionadas** com outras classes por meio da **composição**

Programação Orientada a Objetos

■ Programar no paradigma OO é:

- Identificar os conceitos e entidades do mundo real que serão abstraídas em objetos de software numa linguagem OO: entidades do domínio do problema
 - Se você está implementando um sistema de folha de pagamento, possíveis entidades são: funcionário, salário, contra-cheque, rubrica, vencimentos, descontos, bônus, etc.
 - Se você está implementando um sistema de controle acadêmico, possíveis entidades são: aluno, turma, curso, professor, coordenador, disciplina, semestre letivo etc.
- Definir classes (estrutura + operações) para as entidades do domínio do problema em questão.
 - A criação de uma classe pode usar a definição de classes prévias, criando composições de classes.
- Relacionar classes por meio de herança.
- Entender o mecanismo de vinculação dinâmica da linguagem e aproveitar seu incrível poder.
- Abstrair a sua visão do problema para um conjunto de objetos interagindo.
- Bem-vindo ao mundo da Orientação a Objetos!

Exercício: encapsulamento no mundo real

Enunciado

1. Escolha um objeto do mundo real (ex.: Conta Bancária, Smartphone, Cofre, Elevador, Televisão).
2. Preencha a tabela abaixo indicando **quais atributos e operações devem ser públicas ou privadas**.
3. Justifique a separação escolhida.

Modelo da Tabela

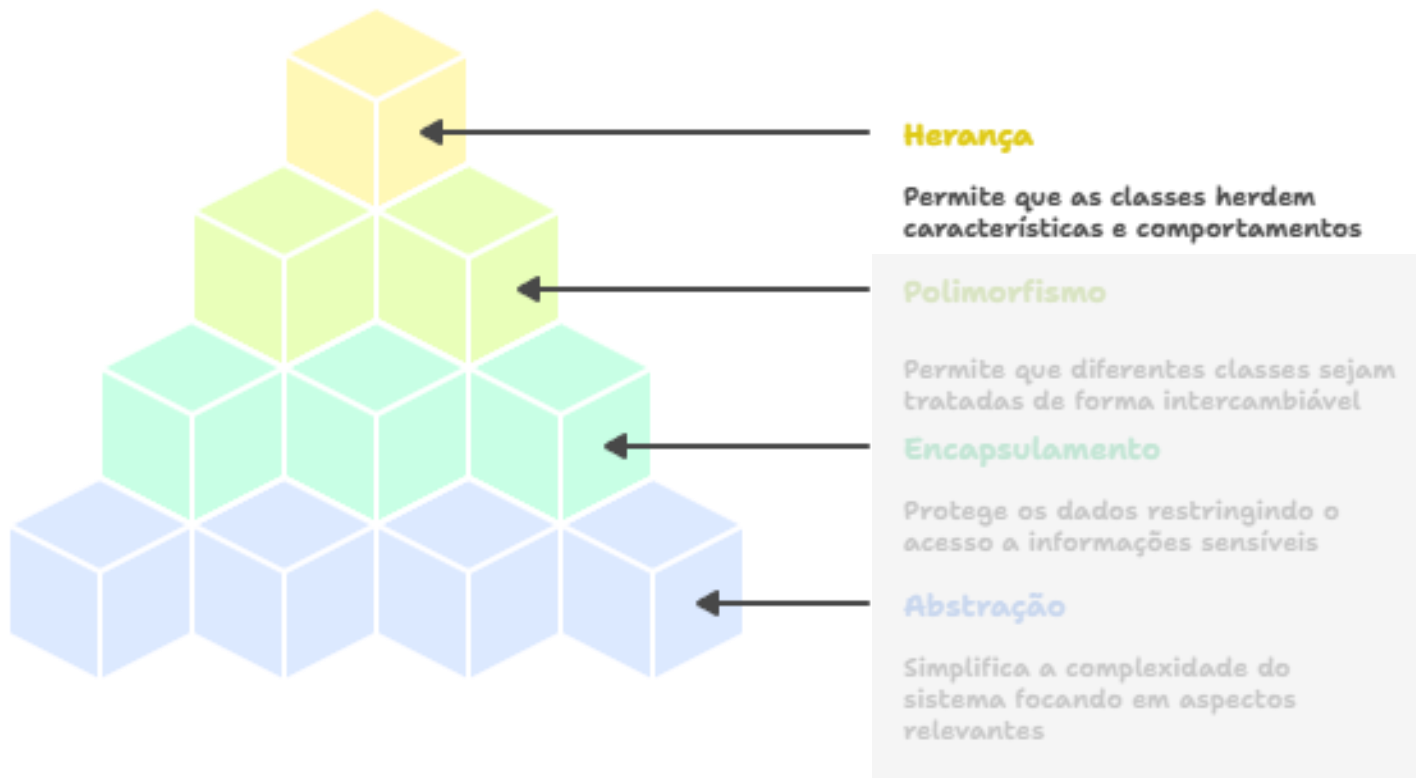
Atributos Públicos	Atributos Privados	Operações Públicas	Operações Privadas

Exemplo preenchido: Conta Bancária

Atributos Públicos	Atributos Privados	Operações Públicas	Operações Privadas
Titular	Saldo	depositar(valor)	descriptografeValores()
Número da conta	Senha	sacar(valor)	conectarBancoCentral()
Agência	Registro de operações	consultarSaldo()	verificarLimiteInterno()

Herança

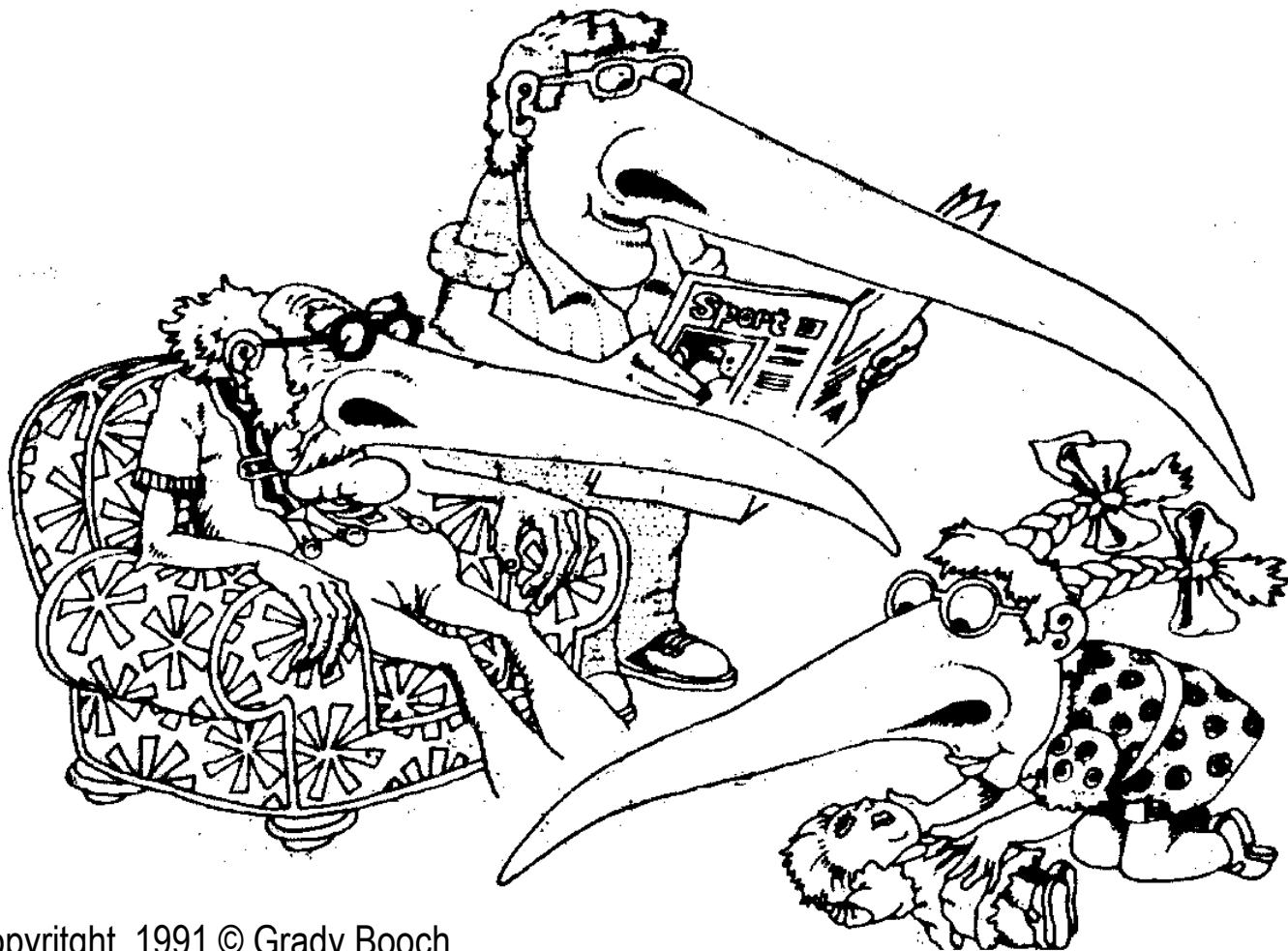
Pilares da Programação Orientada a Objetos



Herança

- A Herança é a contribuição original do paradigma de programação orientado a objetos
- A herança é um relacionamento entre classes
 - Relacionamento do tipo *é um*

Herança



Copyright 1991 © Grady Booch

Herança

- Forma de reutilização de código
 - Propriedades e métodos são herdados, evitando sua duplicação em várias classes similares
- Forma de generalização/especialização de tipos
 - Várias classes podem ser generalizadas em uma classe que concentre todas as características comuns a todas elas
 - Uma classe pode ser especializada em várias outras classes
- Pelo menos duas classes participam da herança

Herança

- Papéis das classes que participam da herança:
 - **Superclasse**: classe a partir da qual são herdadas as propriedades e métodos
 - **Subclasse**: classe que herda as propriedades e métodos da superclasse
- Se uma classe **herda** de outra, então:
 - Os objetos da subclasse possuem todas as propriedades de instância declaradas na superclasse
 - Pode-se enviar para um objeto da subclasse uma mensagem cuja implementação esteja na superclasse

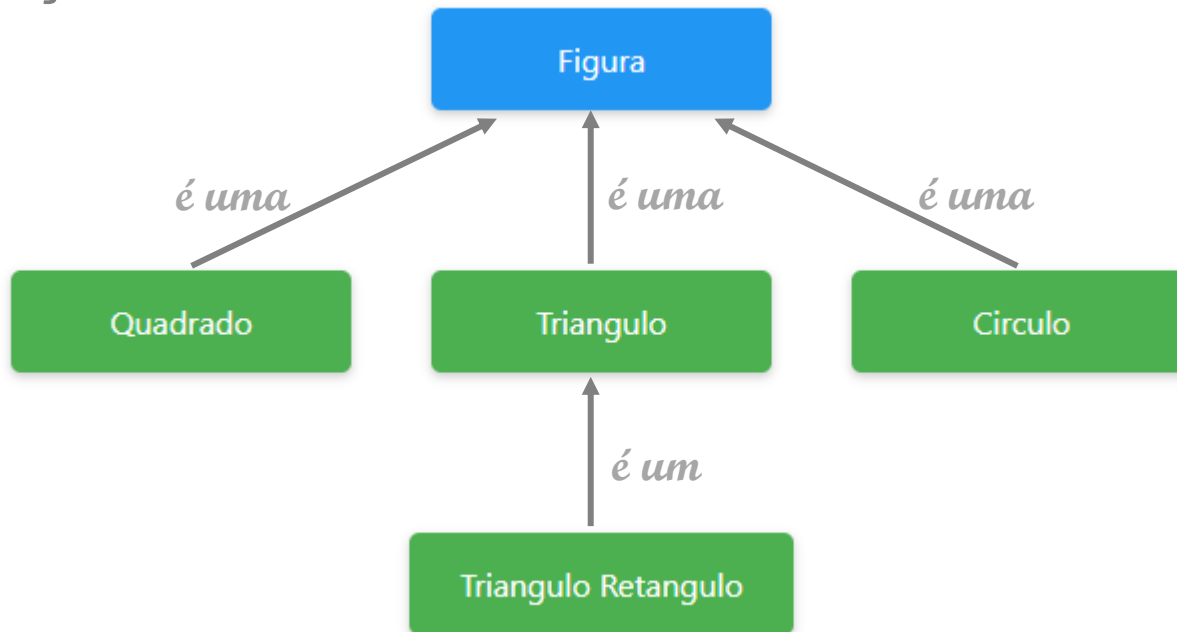
Herança

- Exemplos de relacionamentos de herança no mundo real:
 - Um carro *é um* veículo
 - Um gerente *é um* funcionário
 - Um homem *é um* mamífero
 - Um mamífero *é um* animal
- Na POO a herança não é um relacionamento entre objetos, mas entre as classes destes objetos
 - No mundo real, a herança algumas vezes é uma relação entre seres (o filho herda dos pais)

Herança

- Definindo uma hierarquia de conceitos com a herança:

Especialização



Generalização

Herança: exercício

Enunciado

1. Escolha um domínio do mundo real

Pode ser relacionado a **transportes, animais, profissões, aparelhos eletrônicos** ou outro tema de interesse.

2. Identifique uma entidade “pai” (superclasse)

Esta será a **classe base**, contendo propriedades e operações comuns.

- Exemplo: **Veículo**

3. Crie pelo menos duas entidades “filhas” (subclasses)

Essas herdam características da classe pai e podem adicionar novas propriedades ou operações específicas.

- Exemplo: **Carro e Bicicleta**

4. Destaque o que é comum e o que é específico

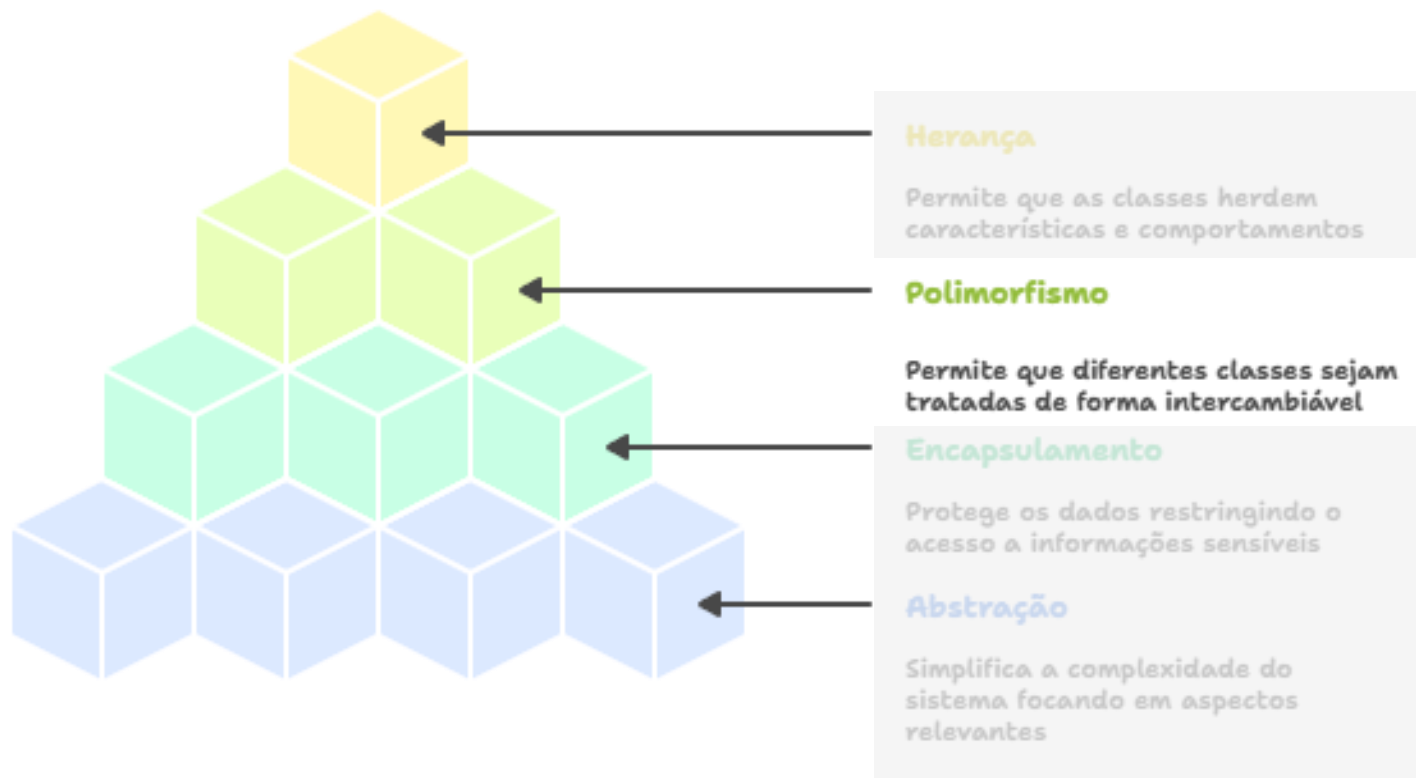
- Comum a todos os veículos: cor, capacidade, acelerar()
- Específico do carro: número de portas
- Específico da bicicleta: tipo de marcha

5. Explique a relação de herança criada

Mostre como as subclasses aproveitam as características da superclasse e apenas adicionam ou especializam comportamentos.

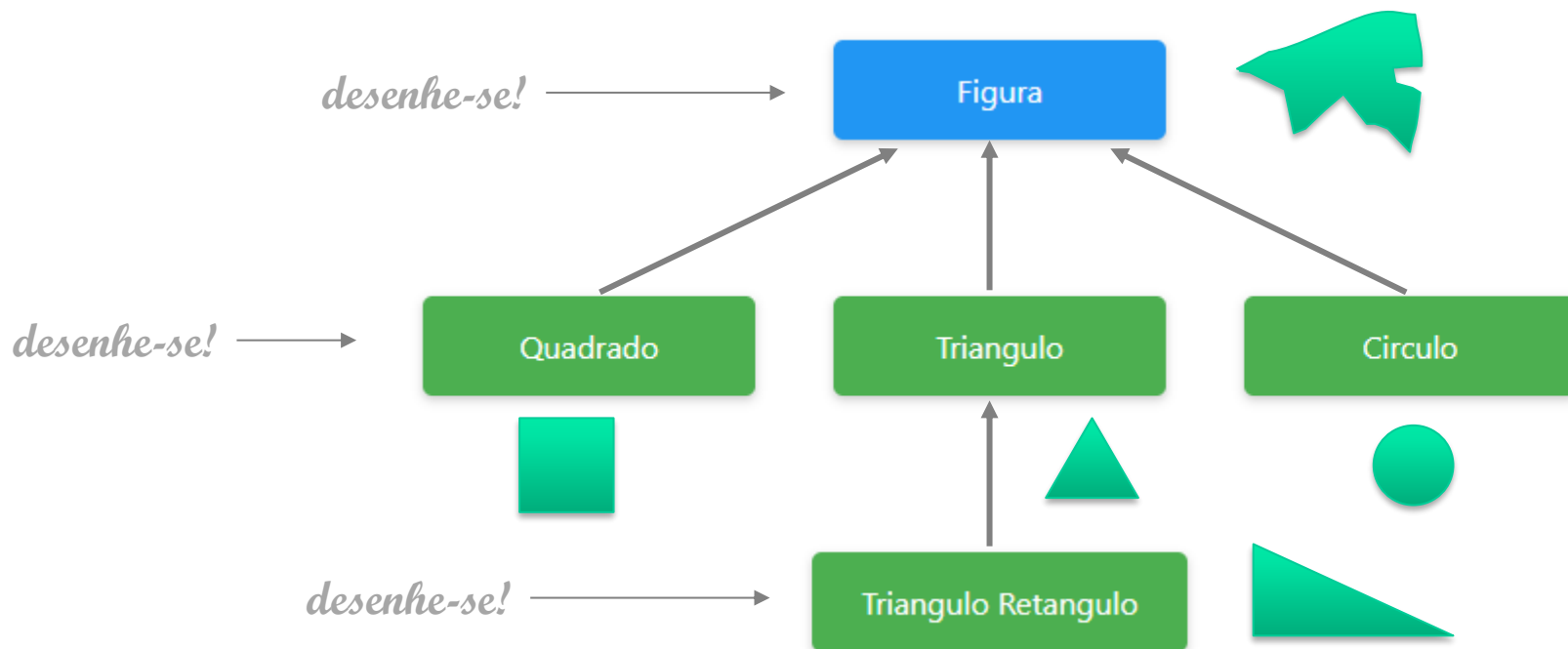
Polimorfismo

Pilares da Programação Orientada a Objetos

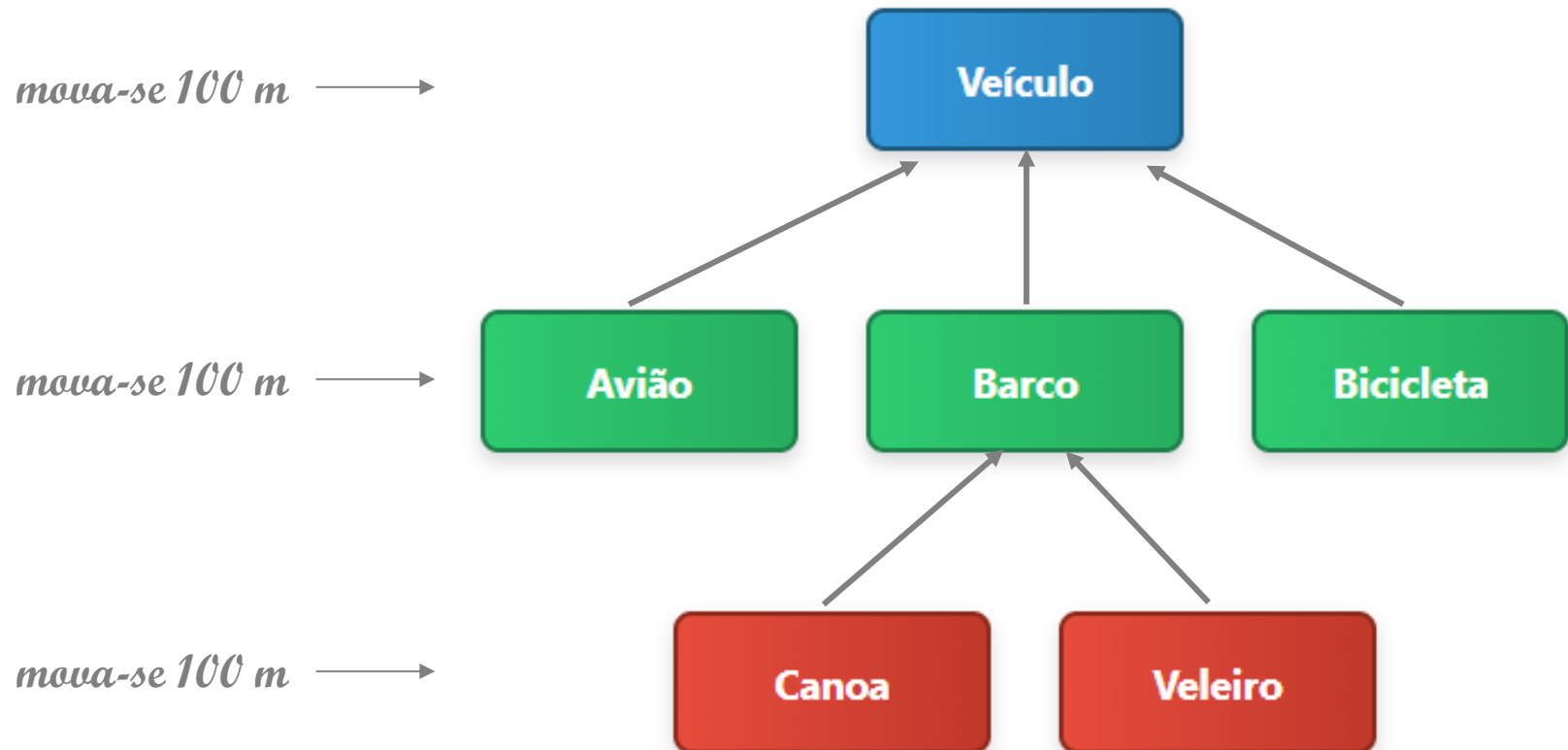


Polimorfismo

- É a habilidade de objetos de classes diferentes responderem à mesma mensagem de formas diferentes.
 - **Poli** (muitos) + **morphos** (forma) = *múltiplas formas*

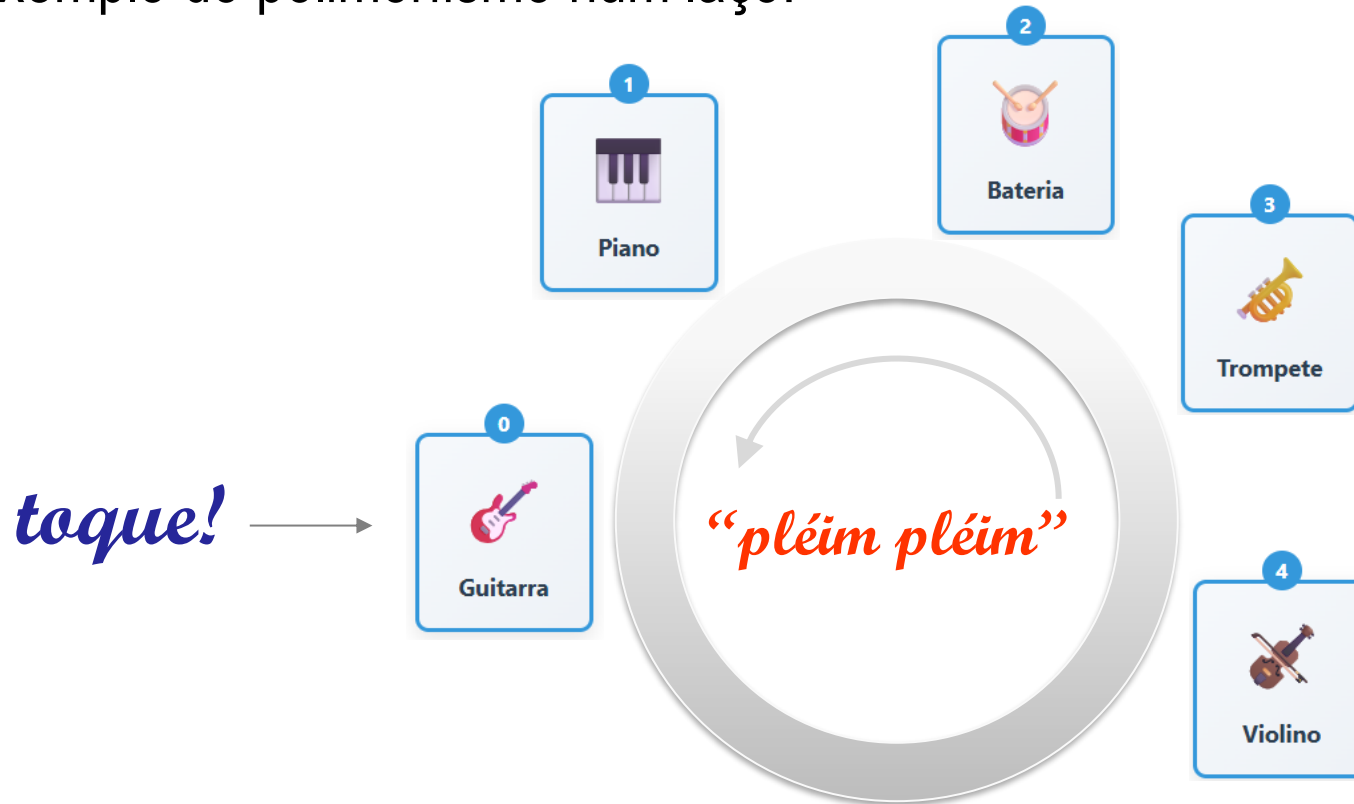


Polimorfismo



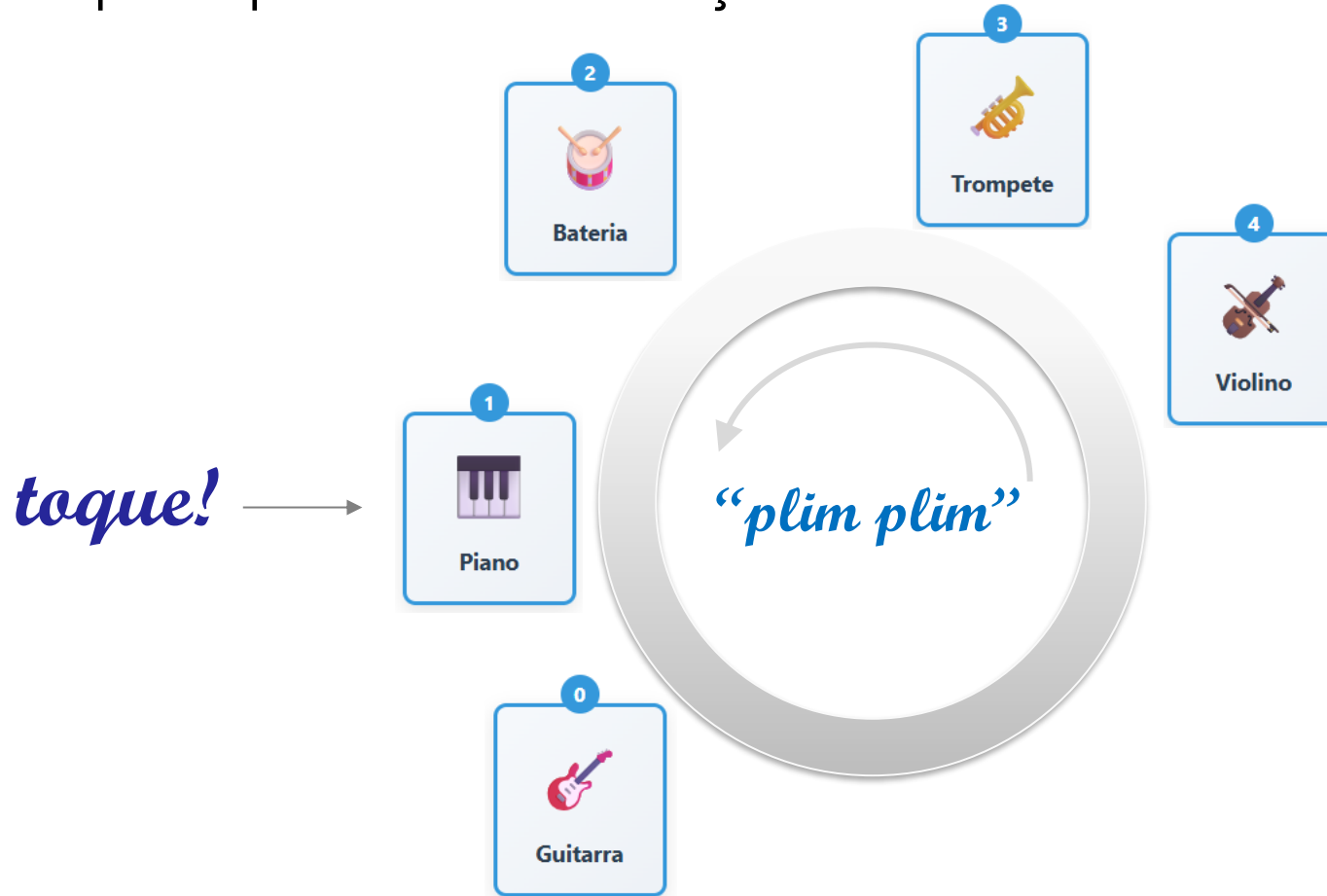
Polimorfismo

- Exemplo do polimorfismo num laço:



Polimorfismo

- Exemplo do polimorfismo num laço:



Polimorfismo

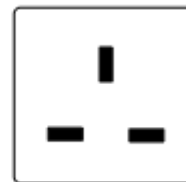
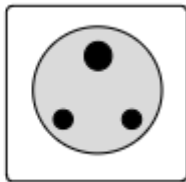
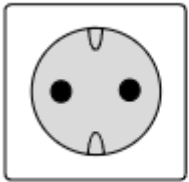
- Exemplo do polimorfismo num laço:



Polimorfismo

- O polimorfismo também pode ser a característica que um objeto tem de apresentar múltiplas formas

Tomadas de tipos diferentes



Tomada polimórfica



Polimorfismo

■ Sem polimorfismo:

array de inteiros

8	-6	0	-1	3
---	----	---	----	---



array de inteiros

8	-6	0	"poo"	3
---	----	---	-------	---



■ Com polimorfismo:

array de objetos

2.1	'f'		"poo"	3
-----	-----	---	-------	---



Polimorfismo: exercício

Enunciado

1. Escolha um domínio do mundo real

Pode ser de áreas como **instrumentos musicais**, **meios de transporte**, **animais**, **eletrodomésticos**, entre outros.

2. Defina uma superclasse com uma operação comum

Essa operação deve ser interpretada de formas diferentes por cada subclasse.

- Exemplo: **InstrumentoMusical** com a operação **tocar()**.

3. Crie pelo menos duas subclasses

Cada uma deve implementar a operação comum de forma diferente.

- Exemplo: **Violão** → **tocar()** produz acordes de corda.
- **Bateria** → **tocar()** produz batidas de percussão.

4. Mostre o comportamento polimórfico

Explique como objetos diferentes podem ser usados de forma uniforme (mesma chamada de operação), mas produzem resultados distintos.

5. Descreva um exemplo prático

Imagine um “conjunto de objetos” (ex.: uma orquestra com vários instrumentos). Ao chamar a operação comum (ex.: **tocar()**), cada objeto reage à sua maneira, mas todos colaboram para um mesmo objetivo.

Bibliografia

■ Outros sites sobre o assunto:

- http://www.campusprogram.com/reference/en/wikipedia/o/ob/object_oriented_programming_1.html
- http://en.wikipedia.org/wiki/Object-oriented_programming
- <http://jeffsutherland.com/papers/Rans/OOlanguages.pdf>
- <http://www.princeton.edu/~ehuang/OOP/index.html>
- Vídeo com palestra de Alan Kay. "The Computer Revolution Hasn't Happend Yet". Disponível em:
<http://www.educause.edu/conference/e98/webcast98.html>